ruby/n_plus_one_example.rb

```
# Show SQL if not already enabled in bin/rails console
ActiveRecord::Base.logger = Logger.new(STDOUT)

Vehicle.all.each do |vehicle|
  vehicle.vehicle_reservations.count
end; nil
```

The count of vehicle_reservations for a Vehicle is queried for every Vehicle.

The SQL queries will look like these:

ruby/n_plus_one_queries.rb

```
Vehicle Load (2.1ms)  \
  SELECT "vehicles".* FROM "vehicles" /*application:Rideshare*/
SELECT COUNT(*) ... WHERE "vehicle_reservations"."vehicle_id" = 1
SELECT COUNT(*) ... WHERE "vehicle_reservations"."vehicle_id" = 2
SELECT COUNT(*) ... WHERE "vehicle_reservations"."vehicle_id" = 3
SELECT COUNT(*) ... WHERE "vehicle_reservations"."vehicle_id" = 4
```

N+1 queries can happen in any Active Record whether it's in the model, views, or serializers. You'll need to discover N+1 instances by monitoring your Rails log or test log, or a team member may spot them during code review.

Manual checks are useful, but automated tools exist that can help you identify N+1 queries as well. The Ruby gem prosopite[3] is an N+1 detection gem and it's been added to Rideshare. Prosopite depends on pg_query[4] for PostgreSQL which was also added.

Let's see if Prosopite picks up this N+1. Run the code again in Rideshare in bin/rails console inside of a Prosopite block. Set Prosopite.rails_logger = true.

ruby/n_plus_one_detection.rb

```
Prosopite.rails_logger = true

Prosopite.scan do
  Vehicle.all.each do |vehicle|
    vehicle.vehicle_reservations.count
  end
end
```

Running this code shows "N+1 queries detected" in the Rails Console. Prosopite can even be configured to detect N+1s in tests and fail the tests.

Now that you can detect N+1 queries manually and automatically, how do you fix them?

3. https://github.com/charkost/prosopite
4. https://github.com/pganalyze/pg_query